

Day 4 实战项目：数据库缓存系统

** 项目结构

```
cache-demo/
+-- redis_cache.py           # Redis缓存封装类
+-- postgres_config.py      # PostgreSQL配置优化
+-- caching_middleware.py   # FastAPI缓存中间件
+-- performance_test.py     # 性能对比测试脚本
+-- requirements.txt        # Python依赖
+-- README.md               # 项目说明
```

*>> 快速开始

☐前置要求

- ☐ 安装Redis # Windows (使用Docker) `docker run -d --name redis -p 6379:6379 redis:alpine`
或使用WSL2/Linux/Mac `sudo apt-get install redis-server`
- ☐ 安装PostgreSQL (可选, 默认使用SQLite) # 使用Docker运行PostgreSQL `docker run -d --name postgres \ -e POSTGRES_PASSWORD=password \ -e POSTGRES_DB=blog \ -p 5432:5432 postgres:15-alpine`
- ☐ 安装Python依赖 `pip install -r requirements.txt`

✓ 功能清单

☐核心功能

- ☐ [x] Redis 缓存读写操作
- ☐ [x] 缓存穿透防护 (空值缓存)
- ☐ [x] 缓存击穿防护 (互斥锁)
- ☐ [x] 缓存雪崩防护 (随机TTL + 多级缓存)
- ☐ [x] FastAPI 缓存中间件集成
- ☐ [x] 性能测试脚本 (有缓存 vs 无缓存对比)

☐技术特性

- ☐ [x] 连接池管理
- ☐ [x] 序列化/反序列化处理
- ☐ [x] TTL 自动过期
- ☐ [x] 批量操作支持

*> 运行示例

☐1. 基础缓存操作

```
from redis_cache import RedisCache

# 初始化连接
cache = RedisCache(host='localhost', port=6379, db=0)

# 设置缓存
cache.set('user:1', {'name': '张三', 'age': 25, ttl=3600})

# 获取缓存
user = cache.get('user:1')
print(user) # {'name': '张三', 'age': 25}
```

☐2. 防护机制演示

```
# 运行性能测试
python performance_test.py
```

** 测试结果示例

```
☐数据库性能测试
=====
```

```
无缓存查询耗时: 125.3ms
有缓存查询耗时: 2.1ms
性能提升: 59.67倍 ☐
```

```
☐所有测试通过!
```

*☐ 相关文档

本项目对应 Day 4 的以下学习内容：

- ☐ PostgreSQL 关系型数据库设计
- ☐ Redis 非关系型数据库应用
- ☐ 缓存策略（穿透/击穿/雪崩防护）
- ☐ 查询优化和索引使用
- ☐ 事务处理

下一步：Day 5：运维与部署实践